

COURSE PROJECT · COMPILER CONSTRUCTION

SIC/XE Assembler

Two-Pass Assembler — Project Plan

Recommended language: Java 17 (C++ acceptable as an alternate)

Team size: Four members (roles defined in Section 7)

Timeline: September 14, 2026 – December submission

Document status: Planning draft

1. Project Goal

Build a two-pass assembler that reads SIC/XE assembly-language source files and produces:

- An object program containing header (`H`), text (`T`), modification (`M`), and end (`E`) records.
- A human-readable assembly listing with addresses, source statements, generated object code, and errors.
- Symbol-table and literal-table reports for debugging and demonstration.

Recommended language: Java 17. Java provides portable file handling, strong data modeling, testing support, and straightforward command-line execution. The design can be implemented in C++ instead if the course requires it; only the syntax and build commands should change.

2. Scope and Success Criteria

Required SIC/XE Support

1. Source-line fields: optional label, operation, operand, comments, and blank lines.
2. Directives: `START`, `END`, `BYTE`, `WORD`, `RESB`, `RESW`, `BASE`, `NOBASE`, `LTORG`, and `EQU`.
3. Symbol definitions and forward references.
4. Literal operands such as `=C'EOF'` and `=X'F1'`.
5. Instruction formats 1, 2, 3, and 4.
6. Addressing modes: simple, immediate (`#`), indirect (`@`), indexed (`,X`), PC-relative, base-relative, and extended (`+`) format.
7. Object-program records, including modification records for relocatable format-4 addresses.
8. Clear errors for invalid opcodes, duplicate/undefined symbols, malformed constants, invalid registers, range errors, and bad addressing combinations.

Completion Definition

The project is complete when all six test programs generate expected addresses and object code, valid object records, understandable listing/error output, and a short user guide explaining how to build and run the assembler.

3. Proposed Design

Command-Line Interface

```
java -jar sicxe-assembler.jar input.asm --out output/
```

The output directory should contain:

<code>input.obj</code>	SIC/XE object program
<code>input.lst</code>	Source, addresses, object code, and diagnostics
<code>input.sym</code>	Sorted symbol table
<code>input.lit</code>	Literal table
<code>input.int</code>	Pass-1 intermediate representation (debugging aid)

Module Layout

<code>src/</code>	
<code>Main.java</code>	CLI and file orchestration
<code>parser/</code>	
<code>SourceParser.java</code>	Parses source into <code>SourceLine</code> objects
<code>OperandParser.java</code>	Parses addressing prefixes, index suffix, literals
<code>model/</code>	
<code>SourceLine.java</code>	Parsed source statement and original line data
<code>IntermediateLine.java</code>	Pass-1 address, block, and source information
<code>Symbol.java</code>	
<code>Literal.java</code>	
<code>Instruction.java</code>	
<code>ObjectProgram.java</code>	
<code>tables/</code>	
<code>OpcodeTable.java</code>	SIC/XE instruction names, opcode, valid formats
<code>RegisterTable.java</code>	
<code>SymbolTable.java</code>	
<code>LiteralTable.java</code>	
<code>pass/</code>	
<code>PassOne.java</code>	Addresses, symbols, literals, intermediate output
<code>PassTwo.java</code>	Object code and object-record generation
<code>output/</code>	
<code>ListingWriter.java</code>	
<code>ObjectWriter.java</code>	

ReportWriter.java	
diagnostics/	
Diagnostic.java	
DiagnosticCollector.java	
tests/	
programs/	Six .asm integration inputs
expected/	Expected .obj/.lst excerpts and symbol values
unit/	Parser and encoding unit tests
docs/	
USER_GUIDE.md	
DESIGN.md	

Core Data

Structure	Essential contents
Opcode table	mnemonic, opcode byte, permitted formats
Symbol table	name, address/value, relocatable/absolute flag, defined line
Literal table	literal spelling, bytes, assigned address, emitted status
Intermediate line	original source, parsed fields, address, generated bytes, diagnostics
Text-record builder	start address, accumulated bytes, 30-byte maximum

4. Assembly Workflow

Pass 1: Analyze and Assign Addresses

1. Read each source line and retain its original text and line number.
2. Parse label, opcode, operand, and comment; report syntax errors without aborting the whole run.
3. Initialize the location counter from `START` ; otherwise start at zero.
4. Validate instructions/directives and compute each statement's byte size.
5. Add labels to the symbol table, detecting duplicates.
6. Collect literals when operands use `=...` ; allocate pending literals at `LTORG` and `END` .
7. Write each resolved statement to the intermediate representation.
8. Determine program name, start address, execution address, and total length.

Pass 2: Encode and Emit Records

1. Read the intermediate representation and resolved symbol/literal tables.
2. Encode directive constants (`BYTE` and `WORD`).
3. Encode instruction formats:
 - Format 1: opcode only.
 - Format 2: opcode plus validated register/numeric fields.
 - Formats 3/4: set `n i x b p e` flags, calculate displacement/address, and detect overflow.
4. Use PC-relative addressing when possible; otherwise use configured base-relative addressing; require format 4 when neither fits.
5. Build text records, splitting them at reservation statements, literal pools, address gaps, and the 30-byte record limit.
6. Emit modification records for relocatable fields that need loader adjustment.
7. Write `H` , `T` , `M` , and `E` records, then produce the listing and reports.

5. Test Plan: Six Assembly Programs

Each test should have checked-in source, expected object program, and expected key listing/symbol-table values.

#	Program purpose	Features verified
1	Minimal <code>START</code> / <code>END</code> program	Header/end records, format 3, simple symbol
2	Data-definition program	<code>BYTE</code> character/hex constants, <code>WORD</code> , <code>RESB</code> , <code>RESW</code> , text-record breaks
3	Addressing-modes program	Simple, immediate, indirect, indexed, PC-relative displacement
4	Base and extended program	<code>BASE</code> , <code>NOBASE</code> , base-relative encoding, format 4, modification records
5	Literals and forward references	<code>LTORG</code> , <code>=C</code> , <code>=X</code> , literal placement, unresolved-then-resolved symbols
6	Error-handling program	Invalid opcode/register, duplicate label, undefined symbol, malformed constant, displacement overflow

For tests 1-5, compare generated `.obj` files exactly against expected files. For test 6, compare expected diagnostic codes/messages and confirm no invalid object code is silently emitted. Add focused unit tests for parsing, numeric/hex conversion, each instruction format, and text-record splitting.

6. Timeline and Milestones

Assumes project work begins the week of **September 14, 2026** and is submitted in December. The last week is intentionally reserved for contingency and submission work.

Dates	Milestone	Deliverable
Sep 14-20	Project setup and SIC/XE research	Repository/folders, Java build command, opcode/register tables, project requirements
Sep 21-27	Design freeze	Class diagram, input/output formats, six test sources drafted, Pass-1/Pass-2 interfaces agreed
Sep 28-Oct 4	Parser and diagnostics	Source parser, operand parser, line-numbered errors, parser unit tests
Oct 5-11	Pass 1 foundations	Location counter, directives, symbol table, intermediate listing
Oct 12-18	Pass 1 complete	Forward references, literals, <code>LTORG</code> , <code>EQU</code> ; tests 1, 2, and 5 passing for Pass 1
Oct 19-25	Pass 2 instruction encoding	Formats 1/2/3, flags, PC-relative and indexed addressing
Oct 26-Nov 1	Advanced encoding	<code>BASE / NOBASE</code> , format 4, modification records
Nov 2-8	Object-program writer	Correct <code>H/T/M/E</code> records, 30-byte text record splitting; tests 1-4 passing
Nov 9-15	Error handling and reports	Listing, symbol/literal reports, comprehensive diagnostics; test 6 passing
Nov 16-22	Integration and regression	All six programs automated; compare expected outputs; fix defects
Nov 23-29	Documentation and demo	User guide, design explanation, sample walkthrough, presentation/demo script
Nov 30-Dec 6	Release candidate	Clean build, final output review, peer/instructor feedback incorporated
Dec 7-deadline	Submission buffer	Final packaging, backup, and submission

7. Task Division

Use the following ownership for a four-person team. For a smaller team, combine adjacent roles while preserving the listed interfaces and review points.

Role	Primary responsibility	Milestone contribution
Member A: coordinator/parser	Requirements, opcode/register data, parser, CLI, diagnostics contract	Sep 14-Oct 4; coordinates integration
Member B: Pass 1	Location management, directives, symbol/literal tables, intermediate representation	Oct 5-18
Member C: Pass 2	Instruction encoder, addressing calculations, relocation/modification records	Oct 19-Nov 1
Member D: output/quality	Object/listing/report writers, expected outputs, automated tests, documentation/demo	Continuous; leads Nov 2-Dec

Shared Responsibilities

1. Hold one short weekly integration meeting and merge only after running the six-program suite.
2. Review another member's code before integration, especially interfaces between parser, Pass 1, and Pass 2.
3. Keep a defect list with source line, expected behavior, actual behavior, owner, and resolution.
4. Every member explains one test program and one assembler component in the final demonstration.

8. Weekly Working Routine

1. **Monday:** choose the week's deliverable and assign concrete tasks.
2. **Midweek:** integrate partial work; run the current automated tests; record defects.
3. **Friday/Sunday:** demonstrate the milestone output, update documentation, and re-plan the following week.

Maintain a small issue board with `To do`, `In progress`, `Review`, and `Done` columns. No feature should enter `Done` until a representative assembly program and expected output cover it.

9. Risks and Mitigations

Risk	Mitigation
Format-3 displacement mistakes	Unit-test positive/negative PC-relative offsets and base-relative fallback independently.
Invalid object-record boundaries	Centralize text-record construction and test 30-byte, gap, and reservation boundaries.
Team integration conflicts	Define <code>SourceLine</code> , <code>IntermediateLine</code> , and diagnostic interfaces before Pass 1 starts.
Late discovery of missing features	Draft all six programs during design week and use them as the implementation checklist.
Schedule slip	Keep November 30 onward free of new features; prioritize correct core assembly over optional enhancements.

10. Deliverables Checklist

- Source code and build/run instructions.
- Opcode and register tables with SIC/XE formats.
- Object program, listing, symbol-table, and literal-table outputs.
- Six source test programs with expected output files.
- Automated test command and passing results.
- Design document describing two-pass processing and addressing-mode calculations.
- User guide and final demonstration materials.